



PROGRAMMING ASSIGNMENT

```
# Python
def solve(problem):
    think()
    code()
    debug()
    return success

for i in range(0, infinite):
    try:
        learn()
        improve()
    except Error:
        pass
    else:
        succeed()
```



NAME : *Subhajit Roy*



ROLL NO. : *54*



PROGRAM NAME : *B.Sc.*



SEMESTER : *2nd*



SUBJECT : *CS-DSC-151
Python Programming*

“
CODE.
LEARN.
BUILD.
REPEAT.
”



Table of Contents

1. Lab Exercises — Part 1

- 1.1 Tip & Split Calculator
- 1.2 Number Guessing Game
- 1.3 Vowel Shouter (String Iteration)
- 1.4 Modular Geometry Tool

2. Case Studies — Part 1

- 2.1 Coffee Shop Point-of-Sale
- 2.2 Secure Password Validator
- 2.3 Freelancer's Invoice Generator

3. Exercise Set 1 — `print()` `sep` & `end` Keywords

- 3.1 `sep="$"` and `end=" ... "`
- 3.2 `sep="***"` and `end="—"`
- 3.3 `sep="*", end=newline` & `sep="#", end="—"`
- 3.4 Comparison: `print(True > False)` and `print(True < False)`
- 3.5–3.8 Binary Conversions — decimal, hex, octal
- 3.9–3.10 Decimal Conversions — octal and hexadecimal

4. Exercise Set 2 — Arithmetic Operators

- 4.1 Exponentiation (`**`) — int vs float behaviour
- 4.2 Multiplication (`*`) — int vs float
- 4.3 Division (`/`) — always returns float in Python 3
- 4.4 Floor Division (`//`) — int stays int, float stays float
- 4.5 Floor Division with negative numbers
- 4.6 Modulus (`%`) — int and float operands
- 4.7 Addition (`+`)
- 4.8 Subtraction (`-`)
- 4.9 Expression: `(4**2) + 5%2 - 3*8`
- 4.10 Expression: `(4%2), 5**2, (5+4**3)`
- 4.11 Two variables — arithmetic operations and print string + integer
- 4.12–4.13 Equality comparison (`int==int` and `int==float`)

4.14–4.15 Comparison operators with multiple variables

4.16 Cross-type equality: int, float, and string

5. Exercise Set 3 — Loops & Strings

5.1 while loop — keep asking until user types ‘quit’

5.2 Separate vowels and consonants from user input

5.3 Print uppercase of the entered word

6. Exercise Set 4 — Escape Characters & String Operations

6.1 Print with newline and escape characters

6.2 Count characters in a string without using len()

6.3 Access first three and last two characters (slicing)

6.4 Count occurrences of the first character

6.5 Find the longest word in a sentence

6.6 Exchange first and last characters of each word

6.7 Insert <> symbol in the middle of each word

6.8 Access words with length less than 3

6.9 Reverse the words in a sentence

6.10 Print the index of each character in the string

6.11 Replace all vowels with a specified character

6.12 Remove duplicate characters from a string

7. Exercise Set 5 — Functions

7.1 TypeError: calling function with too many arguments

7.2 NameError: calling function before it is defined

7.3 Execution order with function call between print statements

7.4 Function with a name parameter

7.5 Variable scope — local variable vs outer variable

7.6 Two parameters — positional argument order matters

7.7 Local variable declared inside a function

8. Exercise Set 6 — Tuples

8.1 Unpack a tuple with 7 different element types

8.2 Convert a tuple to a string

8.3 Retrieve repeated (duplicate) elements of a tuple

- 8.4 Convert a list to a tuple
- 8.5 Find the length of a tuple
- 8.6 Reverse a tuple
- 8.7 Convert a tuple of pairs to a dictionary
- 8.8 Compute the sum of all elements of a tuple

9. Exercise Set 7 — Sets

- 9.1 Union and intersection of two sets
- 9.2 Verify subset relationship between two sets
- 9.3 Remove all elements from a set; frozenset is immutable
- 9.4 Compare two sets for equality
- 9.5 Max and min elements from a set and a frozenset

10. Exercise Set 8 — Dictionaries

- 10.1 Concatenate two dictionaries
- 10.2 Create a dictionary where the keys are numbers
- 10.3 Sum all the values in a dictionary
- 10.4 Sort a dictionary based on its keys
- 10.5 Find the maximum value in a dictionary
- 10.6 Print the dictionary in table form
- 10.7 Split a dictionary into two lists (keys and values)

11. Exercise Set 9 — Regular Expressions

- 11.1 Match 'b' followed by zero or one 'a' using ba?
- 11.2 Find sequences of uppercase letter followed by underscore
- 11.3 Find sequences of uppercase letter followed by lowercase letters
- 11.4 Extract words that contain the letter 's'
- 11.5 Extract words that start with a digit or an underscore
- 11.6 Extract words that end with a digit
- 11.7 Extract all substrings from a given string
- 11.8 Print all occurrences and positions of a substring
- 11.9 Replace all spaces with a comma
- 11.10 Extract all five-character words from a string

12. Lab Exercises — Part 3

- 12.1 Find the largest number in a list
- 12.2 Remove duplicate elements while preserving original order
- 12.3 Count even and odd numbers in a list
- 12.4 Reverse a list without using the built-in reverse()
- 12.5 Find the second largest element in a list
- 12.6 Handle multiple exceptions in a single try block
- 12.7 Custom exception class for invalid age input
- 12.8 Encapsulation — BankAccount class with private attributes
- 12.9 Read CSV file, compute stats, write summary report to file
- 12.10 Bar chart of student marks using matplotlib
- 12.11 Abstract base class for shapes — Circle, Rectangle, Triangle
- 12.12 Iterative Binary Search algorithm
- 12.13 Stack — push/pop/peek and postfix expression evaluator
- 12.14 Queue — enqueue/dequeue/front/is_empty/size
- 12.15 Merge Sort — complete implementation and trace
- 12.16 Quick Sort — last element as pivot with partition
- 12.17 Singly Linked List — insert, delete, search, and display

Subject: CS-DSC-151 Python Programming | Semester: 2nd | Student: Subhajit Roy | Roll No.: 54

Lab Exercises — Part 1

Exercise 1 — The “Tip & Split” Calculator

Objective: Practice input(), print(), explicit type conversion, mathematical operators, and string formatting.

PYTHON CODE

```
1 bill          = float(input("Enter total bill: "))
2 tip_percent   = float(input("Enter tip percentage (e.g., 15, 20): ")
3 )
4 people        = int(input("How many people are splitting? "))
5 tip           = bill * (tip_percent / 100)
6 total         = bill + tip
7 per_person    = total / people
8
9 print(f"Each person must pay: ${per_person:.2f}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tip_split.py
Enter total bill: 150.50
Enter tip percentage (e.g., 15, 20): 20
How many people are splitting? 4
Each person must pay: $45.15
```

Exercise 2 — The Number Guessing Game

Objective: Master while loops, if...elif...else statements, and the break keyword.

PYTHON CODE

```
1 secret_number = 73
2
3 while True:
4     guess = int(input("Enter your guess: "))
5     if guess > secret_number:
6         print("Too high! Try again.")
7     elif guess < secret_number:
8         print("Too low! Try again.")
9     else:
10        print("Congratulations! You guessed it right!")
11        break
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 guessing_game.py
Enter your guess: 50
Too low! Try again.
Enter your guess: 90
Too high! Try again.
Enter your guess: 73
Congratulations! You guessed it right!
```

Exercise 3 — Vowel Shouter

Objective: Apply for loops, the continue statement, string methods, and flow control.

PYTHON CODE

```
1 phrase = input("Enter a phrase: ")
2 result = ""
3
4 for char in phrase:
5     if char == " ":
6         continue
7     elif char.lower() in "aeiou":
8         result += char.upper()
9     else:
10        result += char.lower()
11
12 print(result)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 vowel_shouter.py
Enter a phrase: Hello World
hEll0w0rld
```

Exercise 4 — Modular Geometry Tool

Objective: Define custom functions, use positional and default arguments, and integrate flow control.

PYTHON CODE

```
1 def calculate_area(shape, dim1, dim2=0):
2     if shape == "square":
3         return dim1 * dim1
4     elif shape == "rectangle":
5         return dim1 * dim2
6     elif shape == "triangle":
7         return 0.5 * dim1 * dim2
8     else:
9         return "Invalid shape"
10
11 print(calculate_area("square", 5))
12 print(calculate_area("rectangle", 4, 6))
13 print(calculate_area("triangle", 3, 8))
14 print(calculate_area("circle", 7))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 geometry.py
25
24
12.0
Invalid shape
```

Case Studies — Part 1

Case Study 1 — Coffee Shop Point-of-Sale

PYTHON CODE

```
1 name = input("Enter customer name: ")
2 quantity = int(input("Enter quantity of coffees: ")) # integer
3 price = float(input("Enter price per coffee: ")) # float
4
5 subtotal = quantity * price # subtotal before tax
6 tax = subtotal * 0.05 # 5% tax
7 total = subtotal + tax # final total
8
9 print(f"Thank you, {name}! Your total for {quantity} coffees, "
10       f"including tax, is ${total:.2f}.")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 coffee_pos.py
Enter customer name: Roney
Enter quantity of coffees: 3
```


Case Study 2 — Secure Password Validator

PYTHON CODE

```
1 while True:
2     password = input("Enter a new password: ")
3     if password == "quit":
4         print("Process cancelled.")
5         break
6     if len(password) < 8:
7         print("Password too short! Must be at least 8 characters.")
8         continue
9     else:
10        print("Password accepted!")
11        break
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 password_validator
.py
Enter a new password: abc
Password too short! Must be at least 8 characters.
Enter a new password: mypassword123
Password accepted!
```

Case Study 3 — Freelancer's Invoice Generator

PYTHON CODE

```
1 def calculate_pay(hours_worked, hourly_rate, bonus=0.0):
2     if hours_worked > 40:
3         regular_pay = 40 * hourly_rate
4         overtime_pay = (hours_worked - 40) * hourly_rate * 1.5
5     else:
6         regular_pay = hours_worked * hourly_rate
7         overtime_pay = 0
8     return regular_pay + overtime_pay + bonus
9
10 print(f"Pay 1: ${calculate_pay(45, 20, 100):.2f}") #
    positional
11 print(f"Pay 2: ${calculate_pay(hours_worked=38, hourly_rate=25):.2
    f}") # keyword
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 invoice.py  
Pay 1: $1050.00  
Pay 2: $950.00
```

Exercise Set 1 — print() sep & end

Q1. sep="\$" and end=" ... "

PYTHON CODE

```
1 print("this", "is", "to", "test", sep="$", end=" ... ")
2 print("python language")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q1.py
this$is$to$test ... python language
```

Q2. sep="***" and end="----"

PYTHON CODE

```
1 print("this", "is", "to", "test", sep="***", end="----")
2 print("python language")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q2.py
this***is***to***test----python language
```

Q3. Line 1: sep="*", end=newline Line 2: sep="#", end="---"

PYTHON CODE

```
1 print("this", "is", "to", "test", sep="*", end="\n")
2 print("python language", sep="#", end="---")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q3.py
this*is*to*test
python language---
```

Q4. `print(True > False)` and `print(True < False)`**PYTHON CODE**

```
1 print(True > False)
2 print(True < False)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q4.py
True
False
```

Q5–Q8. Binary conversions**PYTHON CODE**

```
1 print(int("1101", 2))           # Q5: binary 1101 -> decimal
2 print(int("1001", 2))           # Q6: binary 1001 -> decimal
3 print(hex(int("1101", 2)))      # Q7: binary 1101 -> hex
4 print(oct(int("1101", 2)))      # Q8: binary 1101 -> octal
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q5_8.py
13
9
0xd
0o15
```

Q9–Q10. Decimal conversions**PYTHON CODE**

```
1 print(oct(145))                 # Q9: decimal 145 -> octal
2 print(hex(123))                 # Q10: decimal 123 -> hex
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q9_10.py
0o221
0x7b
```

Exercise Set 2 — Arithmetic Operators

Q1. Exponentiation — int vs float

PYTHON CODE

```
1 print(3**2)      # int ** int    -> int
2 print(3. **2)    # float ** int  -> float
3 print(3**2.)     # int ** float  -> float
4 print(3. **2.)   # float**float  -> float
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith1.py
9
9.0
9.0
9.0
```

Q2. Multiplication — int vs float

PYTHON CODE

```
1 print(3*2)
2 print(3. *2)
3 print(3*2.)
4 print(3. *2.)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith2.py
6
6.0
6.0
6.0
```

Q3. Division — always float in Python 3

PYTHON CODE

```
1 print(8/2)
2 print(8. /2)
3 print(8/2.)
4 print(8. /2.)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith3.py
4.0
4.0
4.0
4.0
```

Q4. Floor Division

PYTHON CODE

```
1 print(8//2)
2 print(8. //2)
3 print(8//2.)
4 print(8. //2.)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith4.py
4
4.0
4.0
4.0
```

Q5. Floor Division with negatives

PYTHON CODE

```
1 print(-8//2)
2 print(8. //-2)
3 print(-8//2.)
4 print(-8. //-2.)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith5.py
-4
-4.0
-4.0
-4.0
```

Q6. Modulus

PYTHON CODE

```
1 print(8%2)
2 print(8.5%2)
3 print(8%2.5)
4 print(8.5%2.5)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith6.py
0
0.5
0.5
1.0
```

Q7. Addition

PYTHON CODE

```
1 print(8 + 2)
2 print(-8.5 + 2)
3 print(-8 + 2.5)
4 print(8. + 2.5)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith7.py
10
-6.5
-5.5
10.5
```

Q8. Subtraction

PYTHON CODE

```
1 print(8-2)
2 print(-8.5-2)
3 print(-8-2.5)
4 print(8.-2.5)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith8.py
6
-10.5
-10.5
5.5
```

Q9. $(4**2) + 5\%2 - 3*8$

PYTHON CODE

```
1 # 16 + 1 - 24 = -7
2 print((4**2) + 5%2 - 3*8)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith9.py
-7
```

Q10. $(4\%2)$, $5**2$, $(5+4**3)$

PYTHON CODE

```
1 # 0, 25, 69
2 print((4%2), 5**2, (5 + 4**3))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith10.py
0 25 69
```

Q11. Two variables — arithmetic + string + integer

PYTHON CODE

```
1 a = 15
2 b = 4
3 print("sum =", a + b)
4 print("difference =", a - b)
5 print("product =", a * b)
6 print("quotient =", a / b)
7 print("sum=" + str(a + b))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 arith11.py
sum = 19
difference = 11
product = 60
quotient = 3.75
sum=19
```


Q12–Q13. Equality comparison

PYTHON CODE

```
1 print(5==5)      # Q12: int == int
2 print(5==5.)     # Q13: int == float
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q12_13.py
True
True
```

Q14–Q15. Comparison with multiple variables

PYTHON CODE

```
1 x, y, z = 5, 6, 7      # Q14
2 print(x > y)
3 print(x > z)
4
5 x, y, z = 1, 2, 3      # Q15
6 print(x > y)
7 print((z - 2) == x)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q14_15.py
False
False
False
True
```

Q16. Cross-type equality: int, float, string**PYTHON CODE**

```
1 x = 5
2 y = 5.0
3 z = "5"
4 if(x == y):           # int == float -> True
5     print(x)
6 if (x == int(z)):     # int == int -> True
7     print(x)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 q16.py
5
5
```

Exercise Set 3 — Loops & Strings

Q1. while loop — stop on 'quit'

PYTHON CODE

```
1 while True:
2     word = input("Enter a word (type 'quit' to stop): ")
3     if word == "quit":
4         print("Loop terminated.")
5         break
6     print(f"You entered: {word}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 loop1.py
Enter a word (type 'quit' to stop): hello
You entered: hello
Enter a word (type 'quit' to stop): quit
Loop terminated.
```

Q2. Separate vowels and consonants

PYTHON CODE

```
1 word      = input("Enter a word: ")
2 vowels    = ""
3 consonants = ""
4 for ch in word:
5     if ch.lower() in "aeiou":
6         vowels += ch
7     elif ch.isalpha():
8         consonants += ch
9 print("Vowels:      ", vowels)
10 print("Consonants:", consonants)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 vowels_consonants.py
Enter a word: Python
Vowels:      o
Consonants: Pythn
```

Q3. Print uppercase of entered word**PYTHON CODE**

```
1 word = input("Enter a word: ")
2 print(word.upper())
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 uppercase.py
Enter a word: python
PYTHON
```

Exercise Set 4 — Escape Characters & String Operations

Q1. Escape characters and newline

PYTHON CODE

```
1 print("\"This is\\n\"to test\\n\"Python language\"")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 escape1.py
"This is"
"to test"
"Python language"
```

Q2. Count characters without len()

PYTHON CODE

```
1 s = input("Enter a string: ")
2 count = 0
3 for _ in s:
4     count += 1
5 print("Length:", count)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 count_chars.py
Enter a string: Hello
Length: 5
```

Q3. First three and last two characters

PYTHON CODE

```
1 s = input("Enter a string: ")
2 print("First three:", s[:3])
3 print("Last two: ", s[-2:])
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 slice.py
Enter a string: Python
First three: Pyt
Last two: on
```

Q4. Count occurrences of first character

PYTHON CODE

```
1 s = input("Enter a string: ")
2 first_char = s[0]
3 count = 0
4 for ch in s:
5     if ch == first_char:
6         count += 1
7 print(f"'{first_char}' appears {count} time(s)")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 count_first.py
Enter a string: banana
'b' appears 1 time(s)
```

Q5. Find the longest word in a sentence

PYTHON CODE

```
1 sentence = input("Enter a sentence: ")
2 longest = max(sentence.split(), key=len)
3 print("Longest word:", longest)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 longest_word.py
Enter a sentence: The quick brown fox
Longest word: quick
```

Q6. Exchange first and last characters of each word

PYTHON CODE

```
1 sentence = input("Enter a sentence: ")
2 result = []
3 for w in sentence.split():
4     result.append(w[-1] + w[1:-1] + w[0] if len(w) > 1 else w)
5 print(" ".join(result))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 swap_chars.py
Enter a sentence: Hello World
oellH dorlW
```

Q7. Insert <> in the middle of each word**PYTHON CODE**

```
1 sentence = input("Enter a sentence: ")
2 result = []
3 for w in sentence.split():
4     mid = len(w) // 2
5     result.append(w[:mid] + "<>" + w[mid:])
6 print(" ".join(result))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 insert_mid.py
Enter a sentence: Hello World
He<>llo Wo<>rld
```

Q8. Words with length less than 3**PYTHON CODE**

```
1 sentence = input("Enter a sentence: ")
2 print("Short words:", [w for w in sentence.split() if len(w) < 3])
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 short_words.py
Enter a sentence: I am learning Python now
Short words: ['I', 'am']
```

Q9. Reverse words in a sentence**PYTHON CODE**

```
1 sentence = input("Enter a sentence: ")
2 print(" ".join(sentence.split()[::-1]))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 reverse_words.py
Enter a sentence: Hello World Python
Python World Hello
```

Q10. Print index of each character**PYTHON CODE**

```
1 s = input("Enter a string: ")
2 for i, ch in enumerate(s):
3     print(f"Index {i}: {ch}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 char_index.py
Enter a string: Hi
Index 0: H
Index 1: i
```

Q11. Replace vowels with a specified character

PYTHON CODE

```
1 s      = input("Enter a string: ")
2 ch     = input("Replace vowels with: ")
3 result = "".join(ch if c.lower() in "aeiou" else c for c in s)
4 print(result)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 replace_vowels.py
Enter a string: Python
Replace vowels with: *
Pyth*n
```

Q12. Remove duplicate characters

PYTHON CODE

```
1 s      = input("Enter a string: ")
2 result = ""
3 for ch in s:
4     if ch not in result:
5         result += ch
6 print(result)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 remove_dup.py
Enter a string: programming
progamin
```


Exercise Set 5 — Functions

Q1. TypeError: too many arguments

PYTHON CODE

```
1 def test():
2     print("inside function")
3     test("1")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func1.py
TypeError: test() takes 0 positional arguments but 1 was given
```

Q2. NameError: called before definition

PYTHON CODE

```
1 test("1")
2 def test():
3     print("inside function")
4     test()
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func2.py
NameError: name 'test' is not defined
```

Q3. Execution order with function call

PYTHON CODE

```
1 def test():
2     print("inside function")
3 print("main function")
4 test()
5 print("end of main")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func3.py
main function
inside function
end of main
```

Q4. Function with name parameter

PYTHON CODE

```
1 def test(name):
2     print("welcome", name)
3     name = input("enter ur name")
4     test(name)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func4.py
enter ur nameRoney
welcome Roney
```

Q5. Variable scope — local vs outer

PYTHON CODE

```
1 def test(i):
2     print("inside function i=", i)
3     i = i + 1          # local copy only
4 i = int(input("Enter integer:"))
5 print("in main i=", i)
6 test(i)
7 print("after function calling i=", i)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func5.py
Enter integer:5
in main i= 5
inside function i= 5
after function calling i= 5
```

Q6. Two parameters — positional order

PYTHON CODE

```
1 def test(name, surname):
2     print("name:", name, "surname:", surname)
3     test("usharani", "bhimavarapu")
4     test("bhimavarapu", "usharani")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func6.py
name: usharani surname: bhimavarapu
name: bhimavarapu surname: usharani
```

Q7. Local variable inside function

PYTHON CODE

```
1 def test():  
2     i = 10  
3     print(i)  
4 test()
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 func7.py  
10
```

Exercise Set 6 — Tuples

Q1. Unpack 7-element tuple

PYTHON CODE

```
1 t = (1, 3.14, "hello", True, None, [1,2], (3,4))
2 a, b, c, d, e, f, g = t
3 print(a, b, c, d, e, f, g)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple1.py
1 3.14 hello True None [1, 2] (3, 4)
```

Q2. Convert tuple to string

PYTHON CODE

```
1 t = (1, 2, 3, 4)
2 s = str(t)
3 print(s, type(s))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple2.py
(1, 2, 3, 4) <class 'str'>
```

Q3. Retrieve repeated elements

PYTHON CODE

```
1 t = (1, 2, 3, 2, 4, 1, 5, 3)
2 repeated = sorted([x for x in set(t) if t.count(x) > 1])
3 print("Repeated elements:", repeated)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple3.py
Repeated elements: [1, 2, 3]
```

Q4. List to tuple

PYTHON CODE

```
1 t = tuple([10, 20, 30, 40])
2 print(t, type(t))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple4.py
(10, 20, 30, 40) <class 'tuple'>
```

Q5. Length of tuple

PYTHON CODE

```
1 t = (5, 10, 15, 20, 25)
2 print("Length:", len(t))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple5.py
Length: 5
```

Q6. Reverse a tuple

PYTHON CODE

```
1 t = (1, 2, 3, 4, 5)
2 print(t[::-1])
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple6.py
(5, 4, 3, 2, 1)
```

Q7. Tuple to dictionary

PYTHON CODE

```
1 t = (("name","Roney"), ("age",20), ("city","Guwahati"))
2 print(dict(t))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple7.py
{'name': 'Roney', 'age': 20, 'city': 'Guwahati'}
```

Q8. Sum of all elements

PYTHON CODE

```
1 t = (10, 20, 30, 40, 50)
2 print("Sum:", sum(t))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 tuple8.py  
Sum: 150
```

Exercise Set 7 — Sets

Q1. Union and intersection

PYTHON CODE

```
1 a = {1, 2, 3, 4}
2 b = {3, 4, 5, 6}
3 print("Union:      ", a | b)
4 print("Intersection:", a & b)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 set1.py
Union:      {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
```

Q2. Subset check

PYTHON CODE

```
1 a = {1, 2, 3}
2 b = {1, 2, 3, 4, 5}
3 print("a subset of b:", a.issubset(b))
4 print("b subset of a:", b.issubset(a))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 set2.py
a subset of b: True
b subset of a: False
```

Q3. Clear set; frozenset is immutable

PYTHON CODE

```
1 s = {1, 2, 3}
2 s.clear()
3 print("Set:", s)
4 fs = frozenset({4, 5, 6})
5 print("Frozenset (immutable):", fs)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 set3.py
Set: set()
Frozenset (immutable): frozenset({4, 5, 6})
```

Q4. Compare two sets

PYTHON CODE

```
1 a = {1,2,3}; b = {1,2,3}; c = {4,5,6}
2 print("a == b:", a == b)
3 print("a == c:", a == c)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 set4.py
a == b: True
a == c: False
```

Q5. Max and min from set and frozenset

PYTHON CODE

```
1 s = {5, 3, 8, 1, 9}
2 fs = frozenset({10, 20, 15, 5})
3 print("Set max:", max(s), " min:", min(s))
4 print("Frozenset max:", max(fs), " min:", min(fs))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 set5.py
Set max: 9 min: 1
Frozenset max: 20 min: 5
```


Exercise Set 8 — Dictionaries

Q1. Concatenate two dictionaries

PYTHON CODE

```
1 d1 = {"a":1,"b":2}; d2 = {"c":3,"d":4}
2 print(**d1, **d2)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict1.py
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

Q2. Keys are numbers

PYTHON CODE

```
1 d = {1:"one", 2:"two", 3:"three", 4:"four"}
2 print(d)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict2.py
{1: 'one', 2: 'two', 3: 'three', 4: 'four'}
```

Q3. Sum all values

PYTHON CODE

```
1 d = {"a":10, "b":20, "c":30}
2 print("Sum:", sum(d.values()))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict3.py
Sum: 60
```

Q4. Sort by keys

PYTHON CODE

```
1 d = {"banana":3, "apple":5, "cherry":1}
2 print(dict(sorted(d.items())))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict4.py
{'apple': 5, 'banana': 3, 'cherry': 1}
```

Q5. Find max value

PYTHON CODE

```
1 d = {"x":100, "y":50, "z":200}
2 k = max(d, key=d.get)
3 print(f"Max value: {d[k]} (key: '{k}')
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict5.py
Max value: 200 (key: 'z')
```

Q6. Print in table form

PYTHON CODE

```
1 d = {"Name":"Roney", "Age":20, "City":"Guwahati"}
2 print(f"{'Key':<10} {'Value':<15}")
3 print("-"*25)
4 for k, v in d.items():
5     print(f"{k:<10} {str(v):<15}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict6.py
Key      Value
-----
Name      Roney
Age       20
City      Guwahati
```

Q7. Split into two lists

PYTHON CODE

```
1 d = {"a":1, "b":2, "c":3}
2 print("Keys:  ", list(d.keys()))
3 print("Values:", list(d.values()))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 dict7.py
Keys:   ['a', 'b', 'c']
Values: [1, 2, 3]
```

Exercise Set 9 — Regular Expressions

Q1. 'b' followed by zero or one 'a'

PYTHON CODE

```
1 import re
2 print(re.findall(r"ba?", "b ba baa baaa ba b"))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex1.py
['b', 'ba', 'ba', 'ba', 'ba', 'b']
```

Q2. Uppercase letter followed by underscore

PYTHON CODE

```
1 import re
2 text = "A_list B_tree C_node TestCase"
3 matches = re.findall(r"[A-Z]_", text)
4 print(matches)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex2.py
['A_', 'B_', 'C_']
```

Q3. Uppercase + lowercase sequence

PYTHON CODE

```
1 import re
2 print(re.findall(r"[A-Z][a-z]+", "Hello World Python"))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex3.py
['Hello', 'World', 'Python']
```

Q4. Words containing 's'

PYTHON CODE

```
1 import re
2 text = "lass glass cup spoon dish stone"
3 print(re.findall(r"\b\w*s\w*\b", text))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex4.py
['lass', 'glass', 'spoon', 'dish', 'stone']
```

Q5. Words starting with digit or underscore

PYTHON CODE

```
1 import re
2 text = "hello _world 3apples test 9cats _underscore"
3 print(re.findall(r"\b[0-9_]\w*", text))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex5.py
['_world', '3apples', '9cats', '_underscore']
```

Q6. Words ending with a number

PYTHON CODE

```
1 import re
2 print(re.findall(r"\b\w*[0-9]\b", "test1 hello world2 foo3 bar"))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex6.py
['test1', 'world2', 'foo3']
```

Q7. All substrings of a string

PYTHON CODE

```
1 s = "abc"
2 print([s[i:j] for i in range(len(s)) for j in range(i+1, len(s)+1)
        ])
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex7.py
['a', 'ab', 'abc', 'b', 'bc', 'c']
```

Q8. Occurrences and positions of substring

PYTHON CODE

```
1 import re
2 text, sub = "the cat sat on the mat near the bat", "at"
3 for m in re.finditer(sub, text):
4     print(f"Found '{sub}' at position {m.start()}")
5 print("Total:", len(re.findall(sub, text)))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex8.py
Found 'at' at position 5
Found 'at' at position 9
Found 'at' at position 20
Found 'at' at position 33
Total: 4
```

Q9. Replace spaces with comma

PYTHON CODE

```
1 import re
2 print(re.sub(r" ", ",", "hello world python programming"))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex9.py
hello,world,python,programming
```

Q10. Extract five-character words

PYTHON CODE

```
1 import re
2 print(re.findall(r"\b\w{5}\b", "Hello World Python great class bench"))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 regex10.py
['Hello', 'World', 'great', 'class', 'bench']
```

Lab Exercises — Part 3

Q1. Largest number in a list

PYTHON CODE

```
1 lst = [12, 45, 7, 89, 23]
2 print("Largest:", max(lst))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q1.py
Largest: 89
```

Q2. Remove duplicates while keeping original order

PYTHON CODE

```
1 lst = [1, 2, 2, 3, 4, 3, 5]
2 result = []
3 for x in lst:
4     if x not in result:
5         result.append(x)
6 print(result)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q2.py
[1, 2, 3, 4, 5]
```

Q3. Count even and odd numbers

PYTHON CODE

```
1 lst = [10, 15, 22, 33, 40]
2 even = sum(1 for x in lst if x % 2 == 0)
3 odd = len(lst) - even
4 print(f"Even numbers: {even}   Odd numbers: {odd}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q3.py
Even numbers: 3   Odd numbers: 2
```

Q4. Reverse a list without reverse()

PYTHON CODE

```
1 lst = [1, 2, 3, 4, 5]
2 result = []
3 for i in range(len(lst)-1, -1, -1):
4     result.append(lst[i])
5 print(result)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q4.py
[5, 4, 3, 2, 1]
```

Q5. Second largest element

PYTHON CODE

```
1 lst = [10, 45, 67, 89, 54]
2 unique = sorted(set(lst), reverse=True)
3 print("Second largest:", unique[1])
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q5.py
Second largest: 67
```

Q6. Multiple exception handling in one try block

PYTHON CODE

```
1 def test_exceptions():
2     try:
3         val = int(input("Enter a number: "))
4         result = 10 / val
5         with open("nofile.txt") as f:
6             f.read()
7     except ZeroDivisionError:
8         print("Error: Cannot divide by zero!")
9     except ValueError:
10        print("Error: Invalid input, enter an integer!")
11    except FileNotFoundError:
12        print("Error: File not found!")
13
14    test_exceptions()
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q6.py
Enter a number: 0
Error: Cannot divide by zero!
```

Q7. Custom exception for invalid age

PYTHON CODE

```
1 class NotAdultError(Exception):
2     pass
3
4 def check_age(age):
5     if age < 18:
6         raise NotAdultError("Not Adault")
7     print(f"Age {age} is valid. Welcome!")
8
9 try:
10     age = int(input("Enter age: "))
11     check_age(age)
12 except NotAdultError as e:
13     print("Custom Exception:", e)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q7.py
Enter age: 15
Custom Exception: Not Adault

roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q7.py
Enter age: 22
Age 22 is valid. Welcome!
```

Q8. Encapsulation — BankAccount class

PYTHON CODE

```
1 class BankAccount:
2     def __init__(self, owner, balance=0):
3         self.__owner = owner          # private
4         self.__balance = balance      # private
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.__balance += amount
9             print(f"Deposited ${amount:.2f}")
10
```



```
11     def withdraw(self, amount):
12         if amount > self.__balance:
13             print("Insufficient funds!")
14         else:
15             self.__balance -= amount
16             print(f"Withdrawn ${amount:.2f}")
17
18     def get_balance(self):
19         return self.__balance
20
21     def __str__(self):
22         return f"Account[{self.__owner}] Balance: ${self.__balance:.2f}"
23
24 acc = BankAccount("Roney", 1000)
25 acc.deposit(500)
26 acc.withdraw(200)
27 acc.withdraw(2000)
28 print(acc)
29 print("Balance:", acc.get_balance())
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q8.py
Deposited $500.00
Withdrawn $200.00
Insufficient funds!
Account[Roney] Balance: $1300.00
Balance: 1300
```

Q9. Read CSV, compute stats, write summary report

PYTHON CODE

```
1 import csv, os
2
3 # --- create sample CSV for demo ---
4 sample = [["Name", "Marks"], ["Alice", "85"], ["Bob", "72"],
5           ["Charlie", "91"], ["Diana", "60"], ["Eve", "78"]]
6 with open("students.csv", "w", newline="") as f:
7     csv.writer(f).writerows(sample)
8
9 # --- process ---
10 try:
11     names, marks = [], []
12     with open("students.csv") as f:
13         for row in csv.DictReader(f):
14             names.append(row["Name"])
15             marks.append(int(row["Marks"]))
16
17     avg = sum(marks) / len(marks)
```

```
18     highest = max(marks)
19     lowest  = min(marks)
20
21     with open("summary.txt","w") as f:
22         f.write("=== Student Marks Summary ===\n")
23         for n, m in zip(names, marks):
24             f.write(f"    {n:<10}: {m}\n")
25         f.write(f"\nAverage : {avg:.2f}\n")
26         f.write(f"Highest  : {highest}\n")
27         f.write(f"Lowest   : {lowest}\n")
28
29     print("Summary written to summary.txt")
30     print(f"Average={avg:.2f}    Highest={highest}    Lowest={lowest}"
31           )
32 except FileNotFoundError:
33     print("CSV file not found!")
34 except Exception as e:
35     print("Error:", e)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q9.py
Summary written to summary.txt
Average=77.20    Highest=91    Lowest=60
```

Q10. Bar chart of student marks using matplotlib

PYTHON CODE

```
1 import matplotlib.pyplot as plt
2
3 students = ["Alice", "Bob", "Charlie", "Diana", "Eve"]
4 marks    = [85, 72, 91, 60, 78]
5
6 plt.figure(figsize=(8, 5))
7 bars = plt.bar(students, marks, color=["#4C72B0", "#DD8452",
8     "#55A868", "#C44E52", "#8172B2"], edgecolor="black")
9
10 for bar, mark in zip(bars, marks):
11     plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
12         0.8,
13         str(mark), ha="center", va="bottom", fontweight="bold"
14         )
15
16 plt.title("Student Marks", fontsize=14, fontweight="bold")
17 plt.xlabel("Students")
18 plt.ylabel("Marks")
19 plt.ylim(0, 110)
20 plt.tight_layout()
21 plt.savefig("marks_chart.png")
```

```
20 plt.show()
21 print("Chart saved as marks_chart.png")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q10.py
Chart saved as marks_chart.png
```

Q11. Abstract base class for shapes

PYTHON CODE

```
1 from abc import ABC, abstractmethod
2 import math
3
4 class Shape(ABC):
5     @abstractmethod
6     def area(self):
7         pass
8
9     @abstractmethod
10    def perimeter(self):
11        pass
12
13    def describe(self):
14        print(f"{self.__class__.__name__}: "
15              f"area={self.area():.2f}, perimeter={self.perimeter():.2f}")
16
17    class Circle(Shape):
18        def __init__(self, r):
19            self.r = r
20        def area(self):
21            return math.pi * self.r ** 2
22        def perimeter(self):
23            return 2 * math.pi * self.r
24
25    class Rectangle(Shape):
26        def __init__(self, w, h):
27            self.w, self.h = w, h
28        def area(self):
29            return self.w * self.h
30        def perimeter(self):
31            return 2 * (self.w + self.h)
32
33    class Triangle(Shape):
34        def __init__(self, a, b, c):
35            self.a, self.b, self.c = a, b, c
36        def area(self):
37            s = (self.a + self.b + self.c) / 2
38            return math.sqrt(s*(s-self.a)*(s-self.b)*(s-self.c))
```

```
39     def perimeter(self):
40         return self.a + self.b + self.c
41
42     for shape in [Circle(5), Rectangle(4, 6), Triangle(3, 4, 5)]:
43         shape.describe()
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q11.py
Circle: area=78.54, perimeter=31.42
Rectangle: area=24.00, perimeter=20.00
Triangle: area=6.00, perimeter=12.00
```

Q12. Iterative Binary Search

PYTHON CODE

```
1  def binary_search(arr, target):
2      low, high = 0, len(arr) - 1
3      while low <= high:
4          mid = (low + high) // 2
5          if arr[mid] == target:
6              return mid
7          elif arr[mid] < target:
8              low = mid + 1
9          else:
10             high = mid - 1
11     return -1
12
13 arr    = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
14 target = 23
15 idx    = binary_search(arr, target)
16 if idx != -1:
17     print(f"Element {target} found at index {idx}")
18 else:
19     print(f"Element {target} not found")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q12.py
Element 23 found at index 5
```

Q13. Stack — push/pop/peek + postfix evaluation

PYTHON CODE

```
1  class Stack:
2      def __init__(self):
3          self._data = []
4
```

```
5     def push(self, val):
6         self._data.append(val)
7
8     def pop(self):
9         return self._data.pop() if not self.is_empty() else None
10
11    def peek(self):
12        return self._data[-1] if not self.is_empty() else None
13
14    def is_empty(self):
15        return len(self._data) == 0
16
17    def size(self):
18        return len(self._data)
19
20    def eval_postfix(expr):
21        s = Stack()
22        for token in expr.split():
23            if token.lstrip("-").isdigit():
24                s.push(int(token))
25            else:
26                b, a = s.pop(), s.pop()
27                if token == "+": s.push(a + b)
28                elif token == "-": s.push(a - b)
29                elif token == "*": s.push(a * b)
30                elif token == "/": s.push(int(a / b))
31        return s.pop()
32
33    expr = "5 1 2 + 4 * + 3 -"
34    print(f"Postfix: {expr}")
35    print(f"Result: {eval_postfix(expr)}")
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q13.py
Postfix: 5 1 2 + 4 * + 3 -
Result: 14
```

Q14. Queue — enqueue/dequeue/front/is_empty/size

PYTHON CODE

```
1 class Queue:
2     def __init__(self):
3         self._data = []
4
5     def enqueue(self, val):
6         self._data.append(val)
7
8     def dequeue(self):
9         return self._data.pop(0) if not self.is_empty() else None
```

```
10
11     def front(self):
12         return self._data[0] if not self.is_empty() else None
13
14     def is_empty(self):
15         return len(self._data) == 0
16
17     def size(self):
18         return len(self._data)
19
20 q = Queue()
21 q.enqueue(10); q.enqueue(20); q.enqueue(30)
22 print("Front:", q.front())
23 print("Size: ", q.size())
24 print("Dequeue:", q.dequeue())
25 print("Front after dequeue:", q.front())
26 print("Is empty:", q.is_empty())
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q14.py
Front: 10
Size: 3
Dequeue: 10
Front after dequeue: 20
Is empty: False
```

Q15. Merge Sort — implementation + trace on [38,27,43,3,9,82,10]

PYTHON CODE

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            result.append(left[i]); i += 1
15        else:
16            result.append(right[j]); j += 1
17    result.extend(left[i:])
18    result.extend(right[j:])
19    return result
20
```

```
21 arr = [38, 27, 43, 3, 9, 82, 10]
22 print("Original:", arr)
23 print("Sorted:  ", merge_sort(arr))
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q15.py
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted:   [3, 9, 10, 27, 38, 43, 82]
```

Q16. Quick Sort — last element as pivot

PYTHON CODE

```
1 def quick_sort(arr, low, high):
2     if low < high:
3         pi = partition(arr, low, high)
4         quick_sort(arr, low, pi - 1)
5         quick_sort(arr, pi + 1, high)
6
7 def partition(arr, low, high):
8     pivot = arr[high]           # last element as pivot
9     i = low - 1
10    for j in range(low, high):
11        if arr[j] <= pivot:
12            i += 1
13            arr[i], arr[j] = arr[j], arr[i]
14    arr[i+1], arr[high] = arr[high], arr[i+1]
15    return i + 1
16
17 arr = [64, 34, 25, 12, 22, 11, 90]
18 print("Original:", arr)
19 quick_sort(arr, 0, len(arr)-1)
20 print("Sorted:  ", arr)
```

TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q16.py
Original: [64, 34, 25, 12, 22, 11, 90]
Sorted:   [11, 12, 22, 25, 34, 64, 90]
```

Q17. Singly Linked List

PYTHON CODE

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
```

```
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9
10    def insert_at_beginning(self, data):
11        node = Node(data)
12        node.next = self.head
13        self.head = node
14
15    def insert_at_end(self, data):
16        node = Node(data)
17        if not self.head:
18            self.head = node; return
19        cur = self.head
20        while cur.next:
21            cur = cur.next
22        cur.next = node
23
24    def delete_node(self, data):
25        cur = self.head
26        if cur and cur.data == data:
27            self.head = cur.next; return
28        while cur and cur.next:
29            if cur.next.data == data:
30                cur.next = cur.next.next; return
31            cur = cur.next
32
33    def search(self, data):
34        cur, idx = self.head, 0
35        while cur:
36            if cur.data == data:
37                return idx
38            cur, idx = cur.next, idx+1
39        return -1
40
41    def display(self):
42        vals, cur = [], self.head
43        while cur:
44            vals.append(str(cur.data)); cur = cur.next
45        print(" -> ".join(vals) + " -> NULL")
46
47    ll = LinkedList()
48    ll.insert_at_end(10)
49    ll.insert_at_end(20)
50    ll.insert_at_end(30)
51    ll.insert_at_beginning(5)
52    print("List:      ", end=""); ll.display()
53    ll.delete_node(20)
54    print("After del:", end=""); ll.display()
55    print("Search 30:", ll.search(30))
56    print("Search 99:", ll.search(99))
```


TERMINAL OUTPUT

```
roney@fedora:~/Downloads/PYTHON-SUBHAJIT$ python3 p3_q17.py
List:      5 -> 10 -> 20 -> 30 -> NULL
After del:5 -> 10 -> 30 -> NULL
Search 30: 2
Search 99: -1
```